



Notes

Jul 24, 2026

Meeting Jul 24, 2026 at 10:28 IST

Meeting records  Transcript

Summary

Technical discussion detailed AI security guardrails, migration strategies, and testing framework optimizations for autonomous systems.

AI Security and Migration

Discussions focused on implementing robust security guardrails and human-in-the-loop verification processes during test automation migration. Strategies rely on retrieval augmented generation to improve agent accuracy and context awareness.

Autonomous Testing Debates

Participants analyzed the risks of fully autonomous test deployment without human oversight. The team decided to prioritize security checks over full automation to prevent potential compromises.

Technical Testing Protocols

Technical reviews established protocols for debugging CI/CD pipeline failures and managing regression suites. Framework improvements include using specific locators for shadow document object model elements and schema validation for API testing.

Next steps

- ☐ [Ravi Kolte] Research Evaluation Methods: Explore techniques to develop an evaluation framework independently without relying on existing libraries.
- ☐ [Sumit Chhabra] Present Evaluation Model: Showcase a working model evaluation framework during a live session to help the team understand implementation details.
- ☐ [Ravi Kolte] Confirm Session Time: Verify the availability for the 30 minute session between 6 and 6:30.

Details

- **AI Guardrails and Security:** Sumit Chhabra emphasizes that the primary focus of the interview is on implementing AI with robust guardrails to prevent security breaches, such as the accidental release of internal or personally identifiable information ([00:00:04](#)). They stress that candidates must be prepared to discuss specific security checks and human-in-the-loop conditions they have implemented ([00:01:59](#)).
- **Migration Strategy and AI Implementation:** Ravi Kolte describes their current role as migrating test cases from Katalon to Playwright. They utilize GitHub Copilot to assist with script generation and have implemented local agents that operate on the existing framework to support this migration process ([00:03:51](#)).
- **Handling AI Hallucinations:** To address AI hallucinations during script migration, Ravi Kolte explains that they employ an iterative human-in-the-loop verification process, where they personally verify migrations in Visual Studio Code and update the agent's skills based on those reviews ([00:08:36](#)).
- **Improving Agent Accuracy:** To increase accuracy from 65% to 85%, Ravi Kolte suggests using a Retrieval-Augmented Generation (RAG) system, which allows the AI agent to rely on a specific checklist and provide clear responses when it lacks necessary information ([00:10:52](#)). Sumit Chhabra clarifies that any such implementation requires client approval as a proof of concept before it can be integrated into the framework ([00:11:59](#)).

- **LLM Versus Agent Capabilities:** Ravi Kolte distinguishes between a Large Language Model (LLM) and an agent by noting that an LLM provides a one-shot response, whereas an agent combines an LLM with tools and memory to provide context-aware actions, such as generating itineraries with relevant links and asking follow-up questions ([00:13:10](#)).
- **Evaluation Metrics for LLMs:** Sumit Chhabra outlines standard evaluation metrics, including accuracy, relevance, fairness, groundedness, and hallucination. For environments where predefined libraries like DeepEval, OpenEval, or Ragas are unavailable, they propose creating a custom evaluation model using an LLM as a judge, which assesses outputs based on a CSV data model, context, references, and predefined thresholds ([00:15:51](#)).
- **Reviewer Agents for Coverage:** To ensure AI-generated tests do not silently reduce coverage, Ravi Kolte suggests implementing a "Reviewer Agent" that acts as a checkpoint ([00:20:33](#)). This agent performs a "double-eye" verification by comparing the AI's output against established acceptance criteria and security checklists before stakeholders review it ([00:23:17](#)).
- **Autonomous Testing Strategy:** Sumit Chhabra asks for the strategy regarding a fully autonomous AI system that generates, automerges, and deploys test cases into the regression suite without human intervention ([00:26:17](#)). Ravi Kolte disagrees with this approach, citing the need to prioritize security and arguing that such processes require human review to prevent compromised security ([00:27:59](#)).
- **Playwright Technical Concepts:** Ravi Kolte defines Playwright components, stating that the browser refers to the web client, the browser context is an isolated session that does not share cache or data between instances, and the page serves as the main interface for interacting with modules ([00:29:18](#)).
- **Debugging CI/CD Failures:** When tests fail in a CI/CD pipeline despite passing locally, Ravi Kolte recommends using trace viewers and video recordings generated during execution to diagnose the failure ([00:31:18](#)). Sumit Chhabra adds that potential causes include network speed, infrastructure issues, headless mode limitations, parallel worker collisions on shared data, or hardware and security blocks on cloud machines ([00:32:45](#)).
- **Regression Suite Management:** To prevent performance issues, Sumit Chhabra advises isolating flaky or slow tests into nightly buckets, using tags

and groups to separate smoke and regression runs, and executing these tests after every deployment ([00:33:57](#)).

- **API Testing Parameters:** Ravi Kolte explains that Playwright can perform API testing, where one evaluates the response body, status codes, headers, and authorization tokens (such as bearer or OAuth) ([00:36:46](#)).
- **API Schema Validation:** To move beyond simple spot checks in API testing, Sumit Chhabra emphasizes the need for comparing actual output against a grounded data schema to ensure consistency ([00:38:26](#)).
- **Handling Shadow DOM and Iframes:** Ravi Kolte notes that for iframes in Playwright, one can use the "frame locator" method to interact with elements ([00:38:26](#)). Sumit Chhabra adds that for Shadow DOM structures, one must create a parent node and traverse the child nodes or layered roots to capture elements, as standard XPath may not function directly ([00:40:36](#)).

You should review Gemini's notes to make sure they're accurate. [Get tips and learn how Gemini takes notes](#)

How is the quality of **these specific notes**? [Take a short survey](#) to let us know your feedback, including how helpful the notes were for your needs.



Transcript

Jul 24, 2026

Meeting Jul 24, 2026 at 10:28

IST - Transcript

00:00:04

Sumit Chhabra: Okay. Overall uh we have gone through all those two docs which I shared right. Take care. Understood.

Ravi Kolte: almost almost

Sumit Chhabra: Mhm. Cut

Ravi Kolte: uh

Sumit Chhabra: out. Okay. So everything is clear.

Ravi Kolte: cheese boost. deep.

Sumit Chhabra: The the question will be without using DPL and uh RAS or open Ewell. Okay. How will you create evaluation framework on your own? Uh question.

Ravi Kolte: Okay.

Sumit Chhabra: I answered it but I will not give you answer right away. First you need to research. There is a way without even using those predefined libraries.

Ravi Kolte: They

Sumit Chhabra: We can create a kind of evaluation matrix on our

Ravi Kolte: don't

Sumit Chhabra: own denote it's important

Ravi Kolte: take care.

Sumit Chhabra: and mostly your questions will be related to evaluation criteria itself take

Ravi Kolte: Watch out.

Sumit Chhabra: because uh they are more concerned about how you have implemented AI but more about the guardrails of AI that what are the guardrails that you have set uh so that you don't lose any kind of a loophole or there is no

security breach or you are not uh like basically uh releasing your some internal data or PI data to AI models.

00:01:59

Sumit Chhabra: What are the guard rails that you have implemented? What are the security checks? What are the human loop conditions you have implemented? So that's that's the thing.

Ravi Kolte: Okay, let's check this.

Sumit Chhabra: Fine. We have all mangon is here.

Mangesh Shelar: All

Sumit Chhabra: V3 Mesh

Mangesh Shelar: all

Sumit Chhabra: invite because he's also referred today

Ravi Kolte: Uh-huh.

Sumit Chhabra: morning.

Ravi Kolte: Mhm.

Sumit Chhabra: It will be a great advantage for me too. At least I have someone to trust over there. So,

Ravi Kolte: So that's what we're going to

Sumit Chhabra: start

Ravi Kolte: do.

Mangesh Shelar: Office window

Sumit Chhabra: off.

Mangesh Shelar: Friday.

Sumit Chhabra: Nice. Nice. Nice. Take care of let's start.

Ravi Kolte: for my uh show.

Sumit Chhabra: Sure. Okay.

Ravi Kolte: Show

Sumit Chhabra: Quickly brief me about yourself, your technical skill sets and your AI implementations which you have done in past few years and

Ravi Kolte: Sure.

Sumit Chhabra: how you are incorporating uh AI in your automation frameworks.

Ravi Kolte: Sure.

00:03:51

Ravi Kolte: Sum thank you. Uh so Sumit myself Ramen Gaut. I'm having total 7 years of experience in automation testing. uh and currently I'm working on uh the project of duck uh using the caton tool as automation tool and uh uh we have uh means my day-to-day task are something uh related to uh gathering the information from the functional team members and uh analyzing those on the basis of feasible and not feasible test cases for the automation and automating those and sharing the reports with the stakeholders. uh but uh recently as with the change pace of time like we have implemented the AI where we are uh where we received a request from the client that we need to migrate the test cases from the catalon to catalon to preide. Okay. So currently simultaneously we're working on the uh automation of the test cases with the help of GitHub copilot and also simultaneously we're working on the uh migration plan migration plan as well.

Sumit Chhabra: Thank

Ravi Kolte: So we uh with the help of AI that makes us easy uh which for

00:04:52

Sumit Chhabra: you.

Ravi Kolte: which we are you for which the automation we are using uh for which the automation we are using uh the GitHub copilot on the basis of uh on the basis of by writing the prompt correctly just recorrecting the test steps which are shared by the manual team members we are recorrecting it and means uh updating it with the help of data and uh sharing with those prompts with the GitHub uh GitHub copilot and those are automating the test cases for us. Uh this is something related to the automation but uh with the same parallely uh on the migration plan what we are doing we have created our agents uh in which uh as per the framework of uh

Catalon studio and those agents are working 24x7 uh means whenever we are keeping our system on but yeah uh those agents are helping us to migrate.

Sumit Chhabra: So you can't say it's running 24 by 7 because if you have to keep your system on

Ravi Kolte: No, no, it's not. Yeah, we have to keep a system on but yeah those are not 24 by 7.

00:06:01

Ravi Kolte: So uh on on that uh what the agents are migrating whatever

Sumit Chhabra: So it's better to say you have local

Ravi Kolte: information we have shared.

Sumit Chhabra: agents.

Ravi Kolte: Yeah that's correct we have local agents.

Sumit Chhabra: Take

Ravi Kolte: So whatever information we have shared,

Sumit Chhabra: care.

Ravi Kolte: I have planned or I have shared those information on the basis of by updating the uh by sharing the prompts by sharing the data of uh Catalon Studio by sharing the data of uh the framework of Catalon Studio how exactly the Catalon Studio framework is working what things are there in the particular folders. We have shared all the information we have uh and on that basis these those are migrating our test cases to the uh into the playwright format. Currently we have successfully worked on uh smoke scripts and uh the agents which I've created like uh they have uh they have helped to migrate the project uh means other two projects also.

Sumit Chhabra: Mhm.

Ravi Kolte: Uh so yes that's all about me.

Sumit Chhabra: Feedback. You have prompt and you are doing this and you have given a Catalon as a ten instead of saying that say that you have some local agents which you have

00:07:19

Ravi Kolte: Mhm.

Sumit Chhabra: created and trained on the Catalon framework. Accordingly, uh we have a kind of a um skill set and a reference model which is uh treated as a local rack for that particular agent and it helps that particular agent to convert the code or transcript the code from Catalon, Java, Groovy to Typescript including all the checklist which we have already defined in the

Ravi Kolte: Mhm.

Sumit Chhabra: knowledge base and the skill set. I'm just talking to a person who is just newbie to AI.

Ravi Kolte: Okay.

Sumit Chhabra: Okay.

Ravi Kolte: Sure. Sure. Sure. Show sure.

Sumit Chhabra: Okay. Let's move on to the next part. when you say that you are migrating from Catalon to uh Typescript and you are using AI agents for that there must be some hallucination which AI agents also do because I don't think so there will be 100% accuracy when you migrate right so how you are catering or how you are handling those kind of a hallucination so that uh your accuracy rate is at least uh on a bit higher scale

00:08:36

Ravi Kolte: Uh that's that's uh good question actually. So then like and exact how exactly that's being work. Uh AI some of the times like uh help users means you know provides the hallucinated answers to us and uh and how we are tackling that we are providing we are updating the agents on the basis of the information day-to-day. So whenever we received a response or whenever one the script is being you know migrating or one of the code or the test step test line is being migrating. We uh we personally go to the uh visual studio code and verify that particular whatever the migration is being done for one script and verify that uh once if we uh you know uh which comes with a human in loop intervention uh so we are verifying it if any kind of uh any kind of changes be required we just update the

update the skills for that particular agent at the same time and we are again working onto it. So that's how we are handling the uh hallucination part of the agents and

Sumit Chhabra: So can you say you are doing a reiteration journey again and again and uh

00:09:38

Ravi Kolte: uh

Sumit Chhabra: uh it's a kind of a learning exercise for your agent itself until it's completely not trained.

Ravi Kolte: uh Yes.

Sumit Chhabra: So again and again you are basically reworking a lot.

Ravi Kolte: Yes.

Sumit Chhabra: Is there any other way that agent can be have its own learning mechanism as a local learning model which can train itself? Uh before before that uh Mangesh you have a question you have raised

Ravi Kolte: Uh

Mangesh Shelar: Yeah, like uh question like uh we have our local agents like we have

Sumit Chhabra: your

Mangesh Shelar: created a local agents and in that we have set our guard rails for that agent like how uh we need to like which uh like we have used this like we'll be having our GitHub copilot on our framework right so it will like uh it will track our whole framework right and uh on that particular uh framework we have set at our local uh agent in that we have given the guardrails that uh this uh particular framework how we need to move to uh it typescript so in that way we can do like setting up guard

00:10:52

Sumit Chhabra: But it's a checklist,

Mangesh Shelar: rails

Sumit Chhabra: right? That's already a checklist which Ravi has like Ravi has provided to its local agent and despise of

Mangesh Shelar: Mhm.

Sumit Chhabra: that checklist there are a few areas which they might miss right that's why it's hallucinating still because they were not fully aware of like what exactly will be the challenges at the first level of course the accuracy is 65% next level it went to 70%. But how can we directly go from 70 to at least 85% uh with just a single thing? What is the text stack you can use so that you manually don't have to again and again train

Ravi Kolte: So uh we can use the rag system here uh where uh uh the retribal

Sumit Chhabra: it.

Ravi Kolte: augmentation uh generation where we can provide the exact data and you means the AI agent needs to transfer or I'm sorry the migrate migration of these scripts needs to be done on the bas of those checklist only apart from that if any kind of uh situation here comes in while migration the agent should agent should directly tell us that I didn't have an information on That's Right.

00:11:59

Ravi Kolte: So we cannot do that. So that could be done with the help of

Sumit Chhabra: Mhm.

Ravi Kolte: rag.

Sumit Chhabra: Okay. Okay. So uh tell me um how a normal

Ravi Kolte: Uh one second.

Sumit Chhabra: LLM huh take sort of take but still need an

Ravi Kolte: Uh I just want to know will they

Sumit Chhabra: improvisation

Ravi Kolte: ask like why didn't you implemented it yet? means

Sumit Chhabra: I submitted as a proof of concept to the client side because that's We are a client facing uh organization. So without the client approval and without their go we can't uh implement anything directly into the framework

Ravi Kolte: more rag like do we need to explain the architecture of it like or

Sumit Chhabra: until until they don't ask.

Ravi Kolte: There's

Sumit Chhabra: Okay. So tell me what's the difference between a single oneshot LLM call and agent call? How different uh what is the difference between an agent and

Ravi Kolte: okay. Uh so LLM is a large language model which helps with the information.

00:13:10

Sumit Chhabra: basically

Ravi Kolte: what we wanted uh it just provide the oneshot information that's a uh that's a single response which we receive from the uh LLMs uh whereas in comparison to agents agents are combination of LLM plus tools plus memory so suppose for example uh if I ask I want to go on a trip they would just help me with the itonary the LLM will just help with with the itonary that you know uh means I want to go on a trip to uh Shimla. So they will just help me with a simple itonary which will doesn't include any kind of uh uh any kind of flight information or any other details kind of thing. But if you comparison, if you compare that with the uh agents that which includes the LLM plus tools plus uh memory, it will provide me with the links. It mean uh with the hotel links or the flight links and with the information, how many it will ask me a question as well that how many people are going with you, should I update the itenary and all those things.

00:14:17

Ravi Kolte: So that is a major difference between the LLM and uh agents actually.

Sumit Chhabra: So can we say that agent has a capability to plan, think, organize and then response and LLM is a

Ravi Kolte: That's correct.

Sumit Chhabra: on which we have to judge a model and we have a defined threshold it can be as per the market standard like for the accuracy at least the accuracy need to be go above uh 90 to 95% and relevant score should be above 90 uh 95% your hallucination should be less than 5% and these percentages are basically prompted on a screen as a five or 0.95 like like that kind of a scores uh which are prompted and uh the thresholds are basically defined and when we say how we come up to the solution. There are multiple evaluation matrix models which are available. Uh either we can use the DP val, open eval, ragas these kind of multaluation models which can provide us all these uh metrics and their scores by evaluating themselves against the live AI models. Okay. Question. What if you have to create evaluation model without using any predefined libraries available like open eval deep dp raas py p yarch.

00:16:54

Sumit Chhabra: Uh how you can evaluate these models by creating your own evaluation model and how you will define a threshold for all these metrics? Okay, answer

Ravi Kolte: So I I think so like uh how we can we can create

Sumit Chhabra: the

Ravi Kolte: means what what contains in the Rubik's matrix and acceptance criteria and other criterias. So we will create our own uh own scale where you know like uh own scale of metrics that it should have this kind of information only and it should provide with this information only. So we can do we can create our own uh skills uh for that and it should work on that particular skills only. Right.

Sumit Chhabra: It's not a correct one. So for to answer this you can say that uh if we don't have a capability to use the predefined

Ravi Kolte: Okay.

Sumit Chhabra: models then we'll create our own model which will be running on the based of LLM as a judge. Okay. So we'll create we'll we can either have a single model as LM as a judge or we can create a multimodel collaboration flow uh with the LLM as a judge as a base foundation to it.

00:18:06

Sumit Chhabra: So that uh for that what we need we need a kind of a uh CSV or a data model which contains uh prompts their context their references and the uh actual output which we get from the LLM. So based on that uh once we get the actual uh output from the LLM uh during the runtime and the context and the references which we have we uh give all this data to LM as a judge model and that model comes up with all the metrics with their scores and accordingly we bypass that particular we actually compare that particular score against the threshold metrics which we have and at the end in the HTML report or any other report which we have created uh we can showcase this complete metrics along with They're scoring and based on the thresholds we can declare them as a pass or a fail.

Ravi Kolte: Uh one second is like local llm judge right. So it would be a agent or it would be a files emptyd

Sumit Chhabra: uh lm as a judge model is a kind of a uh python script or any type script can be

00:19:06

Ravi Kolte: files.

Sumit Chhabra: also there which uh is calling a open AI uh model either it can be a cloud model open API model uh any model uh which you have referred and using their uh API calls we can provide them these inputs and based on that uh API calls The response will be like there will be one prompt for specific to that and uh that prompt will be having three inputs the context the reference and the actual output. Okay. And the prompt itself which we have given to that LM. So four inputs will be provided based on those four inputs that LLM based on their its skills uh and the prompt which we have already defined somewhere. It will provide us the accuracy, relevance, fairness, groundness, hallucination, faithfulness all the six categories. Okay. and their percentage and based on these percentage we can predict whether it's a pass or a fail and where is the gap

Ravi Kolte: Okay. Take it. Take it.

Sumit Chhabra: actually valuation metrics I can help you out with that I can uh I can give you one session live session I can showcase you I have a running model evaluation model which predict the for then in that

00:20:33

Ravi Kolte: That would be not

Sumit Chhabra: not defend the framework. Yeah.

Ravi Kolte: go.

Sumit Chhabra: Okay. Uh next question. We are left with 8 minutes. Next question. Uh how would you design a kind of a um review checkpoint? Okay. On so that AI

generated tests don't silently reduce your suits coverage. This is the important. question which uh it was a question to me.

Ravi Kolte: Uh, how would you I'm

Sumit Chhabra: How do you design a review checkpoint? Okay,

Ravi Kolte: sorry.

Sumit Chhabra: a kind of a reviewer agent so that AI uh generated test don't silently reduce your test coverage. For example, I'll give you example question example. Uh there was a kind of a user story which came into your plate and your uh test agent uh your test case creation agent created four test cases and as per him he covered uh all the 100% coverage coverage areas which were mentioned in the acceptance criteria in the description. But somehow you noticed that uh one acceptance criteria was missed and this might have done in the earlier two which you have blindly trusted and you have directly automated those test case.

00:21:50

Sumit Chhabra: So now you are in worry that your test suit is not a 100% coverage suit. So what is the criteria that you will define as a review checklist or a review checkpoint so that you make sure that it don't silently miss any coverage.

Ravi Kolte: Okay. Uh so for this one uh so whenever we received a response uh from the AI suppose like uh in your example like we have a once user story for which uh the test cases are designed on the basis of four parts uh sorry or four test cases were designed. Now out of that like one was missed uh one uh one context was missed uh for which which was a important one. Now what you can do we can uh whatever whenever we received a response or whenever the data which we have received from the AI we can review it through the uh external model or or uh the model which uh we have created. uh again one more agent or one more skill file we can go through that uh we can put the response uh through that and we can verify if the test cases are designed in a proper way or whatever the acceptance criteria which we have mentioned in that skill file or whatever acceptance criteria which we have given to that agent or model we can verify through that if that has been meeting uh the requirement which we have mentioned in that that's completely fine if it is not that means something is missing.

00:23:17

Ravi Kolte: So we can put our uh one more check check checklist or one more check uh checkpoint there before putting the response directly to the or sharing the response directly to the stakeholders.

Sumit Chhabra: Okay. So, review agent will be there a double I.

Ravi Kolte: Yeah.

Sumit Chhabra: Okay. But uh don't you think if uh you are using suppose you are using uh GP

Ravi Kolte: Call X numbers.

Mangesh Shelar: So yeah,

Sumit Chhabra: They get

Mangesh Shelar: what about review agent?

Sumit Chhabra: Answer.

Mangesh Shelar: manual. Come on.

Sumit Chhabra: Next question that you said that review agent which is true. I agree to that but on top of it. My counter question says that if you're using suppose GPT 5.6 okay uh your test case creation agent is also using the same uh model

Ravi Kolte: Mhm.

Sumit Chhabra: GP 5.6 six and using that model itself it has uh done a kindly sign

Ravi Kolte: Okay.

Sumit Chhabra: silently released the coverage. Okay, it has silently missed something correct. So what is the shortity that your another reviewer model with the same uh review agent with the same model as a back end will not do the

00:24:53

Ravi Kolte: Uh so means if I if I would like to uh more

Sumit Chhabra: same

Ravi Kolte: iterate on that something kind of retrieval information uh we can use there uh where we would be inputting uh as I said like about the acceptance criteria

uh and uh which could be in comparison with the rag model kind of thing. So I'll strictly follow particularly that information only if that particular information is not there if it is not meeting that criteria it will directly the agent would uh directly tell us that you know it is not following

Sumit Chhabra: Another

Ravi Kolte: the correct procedure actually or particular data is missing. So we can follow the rag kind of rag procedure there. Uh okay.

Sumit Chhabra: reference context.

Ravi Kolte: Okay.

Sumit Chhabra: reference and you will set some checklist guardrails which were missed in that particular agent. And in the review agent, we'll be mostly focusing on the security and the reviewing checklist more. Okay? Which will be a kind of a double eye on top of it. So that before even coming for the human in loop, it's not missing

00:26:17

Ravi Kolte: with the same right.

Sumit Chhabra: anything.

Ravi Kolte: I mean to say same model.

Sumit Chhabra: It's very important. Okay. Suppose

Ravi Kolte: Uh confirmation. Uh normal

Sumit Chhabra: next question. uh suppose I'm your manager and I want a kind of a AI agent implementation in our insurance portfolio which will directly read the test case from a Jira user story or a DevOps user story and uh create the test cases. Okay. Uh create the test cases and also automate and deploy the test cases. Okay. What will be your framework strategy on it and do you agree with my plan first?

Ravi Kolte: Uh, I'm sorry.

Sumit Chhabra: Okay. My question is that suppose I'm a manager and I want a AI agent uh to autogenerate and automerge its own test into the regression suit before a release just to save the time. So do you agree with this approach and if you agree what will be the implementation strategy that you will implement?

Ravi Kolte: Uh we can but we have to uh we can do this but we have to uh it's just about the test cases you're saying right?

00:27:59

Sumit Chhabra: I'm saying that test cases are automated, automerged into the regression build, automerged into the main branch and then uh tested over the CACDI as well without any human intervention. Do you agree with this plan? If yes, how will you implement

Ravi Kolte: No. Uh, no.

Sumit Chhabra: it?

Ravi Kolte: We means I'm not uh I I I means I personally do not agree with this plan because uh that's

Sumit Chhabra: start. That's a fascinating thought.

Ravi Kolte: all.

Sumit Chhabra: No doubt. But yeah, uh the security can't be compromised. That's the important thing. Okay.

Ravi Kolte: Okay.

Sumit Chhabra: Then you say that uh I will push back a little bit and try to convince my

Ravi Kolte: Okay.

Sumit Chhabra: manager that there are few uh checklist or few security guards that we can't miss.

Ravi Kolte: Okay. Take

Sumit Chhabra: directly

Ravi Kolte: care. Okay. Ticker. Sure. Uh uh so so that's a fascinating uh

Sumit Chhabra: start.

Ravi Kolte: thought actually uh means it it includes lot of things which where we come across with the uh uh different thoughts such as like uh agree or not agree.

00:29:18

Ravi Kolte: Uh uh it it it questions on the security uh security part.

Sumit Chhabra: So question

Ravi Kolte: ide

Sumit Chhabra: what are the different design patterns that you are aware of uh when you implement any playwright for a web or API automation?

Ravi Kolte: uh different design patterns. Okay. Mel

Sumit Chhabra: Okay. Single prototype

Ravi Kolte: framework type.

Sumit Chhabra: design. Okay. Uh so let's go with the other

Mangesh Shelar: Right.

Sumit Chhabra: how

Mangesh Shelar: Right. Right. Okay.

Ravi Kolte: Sure.

Sumit Chhabra: is the difference between browser browser context and page.

Ravi Kolte: Uh so browser is is a complete uh web uh the browser which we are using such as like Google Chrome or uh this one. So uh browser context is something like uh we are using the browser same uh same browser I'm sorry the same browser added two I mean uh browser context is something like we are using two browsers but using the same uh parent parent browser. So suppose like uh but they do not share the cache files or do not share the data with each others.

00:31:18

Ravi Kolte: Uh whereas the page is something like uh is is the main method or or the sorry main interface uh main interface which uh helps us to interact with the methods which are designed in the page page

Sumit Chhabra: Okay.

Ravi Kolte: modules.

Sumit Chhabra: Suppose you have created a playwright automation framework. Okay. And you have one test which is intermediately failing on CI/CD run but 100% passing on your local. How will you debug this kind of a scenario?

Ravi Kolte: So test case is failing on the CCD pipeline but it is locally working completely fine. Uh so u I mean we can we can verify it through the uh trace viewers or in the reports which is being generating trace viewers or the videos which are generating over the CICD executions. We can verify with that uh and we can take the actions accordingly over the CIC CICD pipeline.

Sumit Chhabra: How will you trace it, buddy? What is your strategy? What are the key components that you will look into?

Ravi Kolte: Uh key components would be uh where exactly it is being failing.

00:32:45

Ravi Kolte: Uh how it is failing on which step it is failing.

Mangesh Shelar: We start from synchronization issues. Um category bucketing in a

Sumit Chhabra: H keym what are the multiple parameters that can tends to get it failed right that can be speed network speed infra uh then we have a headless mode execution which might not be supporting at that moment take uh parallel workers colliding on the shared

Mangesh Shelar: Yeah, dig.

Sumit Chhabra: data uh we have a kind of a we have multi there can be example of that somebody have might have coded to use a real uh hardware uh kind of a keyboard okay which we need to rephrase the code that's why it's running 100% or that can be also a scenario take care uh there can be a scenario that uh some of the executors or something which is blocked on the CICD run or on the particular cloud machine which we need to make sure that security is enabled to access that particular board, maybe the view port is blocked. There can be multiple scenarios and all these scenarios can be determined on the basis of the videos and the uh uh logs plus

00:33:57

Ravi Kolte: Feels

Sumit Chhabra: snippets.

Mangesh Shelar: feel machine Connectivity problem.

Sumit Chhabra: H possibility.

Ravi Kolte: Sure.

Sumit Chhabra: Okay. How do you design your suit so smoke in sprint and full regression run don't all slow each other down?

Ravi Kolte: Uh, how do you design smoke?

Sumit Chhabra: a regression suit. Uh, sorry. How do you design a framework so your smoke inprint automation and your regression suit don't slow each other down

Ravi Kolte: Uh so if I'm not wrong basically it does something like uh the smoke suit should be uh should be updated as per the regression suit right

Sumit Chhabra: ution tenses with a question.

Ravi Kolte: uh execution uh

Sumit Chhabra: My tags use groups

Ravi Kolte: Uh oh.

Sumit Chhabra: work. passing. Isolate the flaky test or slow test into the nightly bucket. regression nightly smoke like after the every single deployment of the devint. release. Next.

Ravi Kolte: ticket. this cases.

Sumit Chhabra: Okay. Um uh in play right can we do the API automation as well?

00:36:46

Ravi Kolte: Uh yes.

Sumit Chhabra: Okay. So um tell me what are the key parameters when you send an API

Ravi Kolte: Yes.

Sumit Chhabra: via playwright uh HTTP request when it's sent what are the key parameters that you actually look into uh in the response

Ravi Kolte: uh the body of response uh what exactly the JSON file which is being generated with the body of the response which uh it

Sumit Chhabra: only

Ravi Kolte: generates

Sumit Chhabra: status code header authorization key

Ravi Kolte: Let's start

Sumit Chhabra: body.

Ravi Kolte: again.

Sumit Chhabra: Maybe we have authorizations codes like beer tokens or ooth tokens. We have uh header metadata. We have body response. That's correct. We have response status. We have a response code. Okay. Okay.

Ravi Kolte: Huh?

Sumit Chhabra: Uh so how do you validate a kind of a API response uh that it matches the expected output uh or expected schema not just a kind of a spot check for a few

Ravi Kolte: What?

Sumit Chhabra: fields?

Ravi Kolte: Um output.

00:38:26

Ravi Kolte: Uh so again like uh status means we verify with uh schema schemes.

Sumit Chhabra: AP focus question.

Mangesh Shelar: Consistently

Sumit Chhabra: Hey. question and same in API also. If you have to compare a schema with something, you should have a grounded data, right?

Mangesh Shelar: Developers number string.

Sumit Chhabra: for

Ravi Kolte: You can see

Sumit Chhabra: uh so

Ravi Kolte: you.

Sumit Chhabra: how do you test a kind of a DOM structure which is like a shadow DOM or a I frame structure uh in playwright.

Ravi Kolte: So uh for in playwright for uh see I'm not sure about the shadowdom because topic but about the uh about the I frames uh we verify it directly by uh means here is no there is no such concept of switching into the iframe in the playwright but uh we And we can directly play uh uh frame we we use the method as a page dot frame locator something uh sorry not something but page frame locator in which and it will directly uh move to that move to the elements which are there in the iframes a particular iframe which we are going to uh showcase in that frame locator method.

00:40:36

Sumit Chhabra: point this shadow structure I have not yet got a chance to explore in the playwright but yeah I have did it in selenium and over there we create a parent node and inside the chine nodes we try to capture the all the shadow elements of that particular shadow route and accordingly if there are multi- shadow layer uh layering and uh we go onto the uh we try to capture all the roots of the shadow shadow shadow dom and accordingly we can trace down the uh child and this is how the x path will actually work over there otherwise the direct x path will never work and then I

Ravi Kolte: Sure.

Sumit Chhabra: frame explain right at least

Ravi Kolte: Show me.

Sumit Chhabra: Mostly I'm done here. Playright 70% rate still 60 70% CIC mode 60 70% overall rating 60 70% WhatsApp

Ravi Kolte: Um, sure.

Sumit Chhabra: It will help you out. already go through this one too and maybe it will help you out with this and WhatsApp key color.

Ravi Kolte: How many uh we could do?

Mangesh Shelar: Mother.

00:42:58

Sumit Chhabra: or agent or

Mangesh Shelar: Oh,

Sumit Chhabra: agent.

Mangesh Shelar: speech speech.

Sumit Chhabra: Agent routing agent Let's

Mangesh Shelar: This is

Sumit Chhabra: go.

Mangesh Shelar: routing. Delete.

Sumit Chhabra: This is a QA or QA reviewer.

Mangesh Shelar: Mhm.

Sumit Chhabra: This is which means reviewing ase are my subent

Mangesh Shelar: It's a bit more.

Sumit Chhabra: based on the page model and everything based on the checklist and All the criterias which are listed into

Mangesh Shelar: So basically

Sumit Chhabra: iter

Mangesh Shelar: pass data, right?

Sumit Chhabra: It's communicating with all those agents. Once that agent is done with their work, then it go ahead and start with its own.

Mangesh Shelar: First input. I'm good at it.

Sumit Chhabra: confusion

Mangesh Shelar: They

Sumit Chhabra: doesn't matter.

Mangesh Shelar: start

Sumit Chhabra: Controlling agent.

Mangesh Shelar: agents applications. Okay.

Sumit Chhabra: creating

Mangesh Shelar: there. Oh,

Sumit Chhabra: agent full autonomous framework

Mangesh Shelar: four MCPS.

Sumit Chhabra: digit. What? No. No. No. No. Main

Mangesh Shelar: sequence.

Sumit Chhabra: primary

Mangesh Shelar: multi- aent

Sumit Chhabra: multi aent single primary agent focus.

Mangesh Shelar: single.

Sumit Chhabra: Come

Mangesh Shelar: Okay.

Sumit Chhabra: on.

Mangesh Shelar: Proxy

Sumit Chhabra: Mhm.

Mangesh Shelar: problem.

Sumit Chhabra: for better at least we'll get to know what exactly are the questions that are floating over there again. Okay.

Mangesh Shelar: Oh,

Ravi Kolte: I just need to confirm.

Mangesh Shelar: feel This

Sumit Chhabra: Hello.

Ravi Kolte: after

Sumit Chhabra: Can you hear me?

Ravi Kolte: 6 to 6:30. 30 minutes. Notable. Surely go

Sumit Chhabra: will help you out. All the best.

Ravi Kolte: Uh-huh. Sure. Sure. Sure.

Sumit Chhabra: by. Okay.

Ravi Kolte: Yes.

Sumit Chhabra: meeting.

Ravi Kolte: Boo. Yeah. Yeah.

Sumit Chhabra: Thank you.

Ravi Kolte: Okay.

Sumit Chhabra: Bye.

Ravi Kolte: Yes. Bye.

Transcription ended after 00:49:39

This editable transcript was computer generated and might contain errors. People can also change the text after it was created.